

Lab 4 — Vibe Coding

Project: Minesweeper

Repository: SETAPESU26/13_minesweeper

Tool used: ChatGPT

Environment: Windows PowerShell and Python

Chat History

The following conversation documents the prompts, debugging steps, code modifications, testing, and Git commits used during the Minesweeper assignment.

Prompt 1 — Repository Setup and Guidance

"I am working on the assigned GitHub repository https://github.com/SETAPESU26/13_minesweeper. Guide me step by step through the project. I am using Windows PowerShell/Terminal, not VS Code. Help me clone the repository, inspect the files, run the original Python game, implement the required tasks, test the changes, and push the final code to my personal GitHub repository. Give me terminal commands suitable for Windows."

Commands used:

```
cd "C:\Users\Pooja L R\Documents"

git clone https://github.com/SETAPESU26/13_minesweeper.git

cd 13_minesweeper

dir

git status

python --version

python main.py
```

4. Task 1 — Boundary-Safe Traversal

Problem Identified

The neighbour traversal logic in board.py used inclusive upper-bound comparisons. For a board with a given number of rows and columns, valid zero-based indexes range from zero to the dimension minus one. Using `<=` could generate invalid coordinates outside the board.

Prompt 2 — Fix the Boundary Traversal Bug

"I am working on a terminal Python Minesweeper project.

Task 1 requirement: Correct the neighbour traversal so every coordinate visited by reveal and mine-counting logic is a valid board coordinate. Flood-fill must work correctly from corners, edges, and interior cells without exceptions or phantom cells.

The existing code contains:

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols:

    yield nr, nc
```

Analyze the existing code and fix only the boundary traversal issue. Do not rewrite the project or add dependencies. Keep the existing modular structure and behaviour.

Explain the bug, why the inclusive upper bound is incorrect, the corrected condition, and the edge and corner cases to test."

Correction applied in board.py:

```
if 0 <= nr < self.rows and 0 <= nc < self.cols:
```

```
    yield nr, nc
```

Testing commands:

```
python -m py_compile board.py
```

```
python main.py
```

```
git diff board.py
```

```
git status
```

Git commit:

```
git add board.py
```

```
git commit -m "Fix boundary-safe board traversal"
```

Expected result: Neighbour traversal stays within valid board coordinates.

5. Task 2 — Win and Flag Behaviour

Requirements

- Flags should toggle on and off.
- A flagged cell should not accidentally be revealed.
- Invalid actions should not corrupt the board.
- The game should recognize a win when every non-mine cell has been revealed.
- The player should not need to flag every mine to win.

Prompt 3 — Verify and Implement Win and Flag Behaviour

"Analyze the existing board.py and game.py in my Minesweeper project.

Verify the following requirements:

1. Flags can be toggled on and off.
2. Revealing a flagged cell does not change its state.
3. Invalid coordinates and commands do not crash the game or corrupt the board.
4. The game recognizes a win only when every non-mine cell has been revealed.
5. The player does not need to flag every mine to win.

Preserve existing functionality, reuse the existing Board class, and make only the necessary changes. Explain which requirements are already implemented and which need modification. Provide suitable tests."

Verification commands:

```
findstr /n /i "toggle_flag" board.py
```

```
findstr /n /i "won" board.py
```

```
findstr /n /i "def reveal" board.py
```

```
python main.py
```

Testing actions:

```
f 2 2
```

```
f 2 2
```

```
r 0 0
```

```
r 100 100
```

The first two commands test flag toggling. The invalid coordinates should be rejected without crashing the game.

Note: The existing starter code already contained `toggle_flag()` and `won()` methods. These methods were inspected to determine which requirements were already supported.

6. Task 3 — Add Difficulty Modes

Requirements

- Easy, Medium, and Hard difficulty modes.
- Different board dimensions and mine counts for each mode.
- Difficulty selection stored entirely in memory.
- No JSON, CSV, SQLite, or other persistence.
- No additional dependencies.
- Reuse the existing Board class.
- Validate that the mine count is smaller than the total number of board cells.
- Preserve existing reveal and flag commands.

Prompt 4 — Implement Difficulty Modes

"Implement Task 3 in my existing terminal Python Minesweeper project.

Add three difficulty modes: Easy, Medium, and Hard. Each mode must have different board dimensions and mine counts.

Requirements:

1. Mode selection must happen entirely in memory.

2. Do not create JSON, CSV, SQLite, or any other persistence.
3. Do not add external dependencies.
4. Existing reveal and flag commands must continue working.
5. Reuse the existing Board class rather than creating an unrelated implementation.
6. Keep the project modular and understandable.
7. Validate that the mine count is smaller than the number of board cells.
8. Explain the changes before giving the code."

Difficulty configuration used:

Difficulty Dimensions Mines

Easy	6 × 6	6
Medium	8 × 8	12
Hard	10 × 10	20

The configuration was stored in a Python dictionary in game.py.

Testing commands:

```
python -m py_compile game.py
```

```
python main.py
```

Testing procedure:

1. Select Easy using 1.
2. Verify the 6 × 6 board and 6 mines.
3. Restart the game and select Medium using 2.
4. Verify the 8 × 8 board and 12 mines.
5. Restart the game and select Hard using 3.
6. Verify the 10 × 10 board and 20 mines.
7. Enter an invalid difficulty to verify input validation.

Git commit:

```
git add game.py
```

```
git commit -m "Add Minesweeper difficulty modes"
```

7. Task 4 — Action-Level Reveal Feedback

Problem and Requirement

A single player reveal command may trigger an internal flood-fill that visits many cells. The game should provide concise feedback for the player's action rather than printing repeated messages for every internal flood-fill iteration.

Prompt 5 — Implement Action-Level Feedback

"Implement Task 4 in my existing terminal Python Minesweeper project.

Add concise action-level feedback for a player's reveal action.

Requirements:

1. Feedback should correspond to an actual player command.
2. A single reveal command that expands through many zero cells counts as one player action.
3. Internal flood-fill iterations must not produce repeated player-facing messages.
4. Keep feedback concise.
5. Preserve existing game behaviour.
6. Do not rewrite the flood-fill algorithm unnecessarily.
7. Do not add dependencies.
8. Keep the existing modular structure.

Show exactly where the feedback should be generated and explain why it represents a player action rather than an internal iteration."

Implementation concept

The feedback is generated in `game.py`, immediately after the call to `self.board.reveal((r, c))` returns.

```
hit_mine = self.board.reveal((r, c))
print(f"Revealed ({r + 1}, {c + 1}).")
```

```
if hit_mine:
    self.display(reveal_mines=True)
    print("BOOM! You hit a mine.")
    return
```

The board's internal flood-fill remains unchanged. Because the feedback is printed once after the reveal operation completes, a single player command generates one feedback message.

Testing commands:

```
python -m py_compile game.py
```

```
python main.py
```

Example player commands:

r 1 1

f 2 2

f 2 2

q

Expected behaviour: A reveal command produces one concise feedback message, even if multiple cells are revealed internally.

Git commit:

```
git add game.py
```

```
git commit -m "Add action-level reveal feedback"
```

8. Debugging and Validation

During implementation, an indentation error occurred while editing game.py. The file was corrected, and Python's syntax compilation command was used to validate it.

Validation command:

```
python -m py_compile game.py
```

No output indicates that Python did not report a syntax or indentation error.

Other useful Git commands included:

```
git status
```

```
git diff board.py
```

```
git diff game.py
```

```
git diff --check
```

```
git log --oneline -5
```

The `__pycache__` directory was recognized as Python-generated cache and was not intended to be included in the source submission.

9. Git Version Control

Separate commits were created for the implemented changes.

Commit history recorded during the work:

```
d6a4a09 Add action-level reveal feedback
```

```
55732d4 Add Minesweeper difficulty modes
```

```
bab6096 Fix boundary-safe board traversal
```

```
217dd03 Update scenario title to Scenario 13
```

Task 1, Task 3, and Task 4 each have separate implementation commits. The original starter code already contained the core flag-toggle and win-checking methods, which were inspected during Task 2 verification.

The repository was intended to be pushed to a personal GitHub repository rather than submitting a pull request to the assigned SETAPESU26 repository.

10. Final Testing Checklist

- Original game runs.
- Boundary traversal works for corners, edges, and interior cells.
- Invalid coordinates are rejected.
- Flag toggling works.
- Revealing a flagged cell is prevented.
- Win condition is verified.
- Easy mode works.
- Medium mode works.
- Hard mode works.
- One reveal action produces one feedback message.
- Python syntax validation succeeds.
- Git history contains separate implementation commits.
- Final code is pushed to the personal GitHub repository.
- Before and after videos are recorded.

11. Submission Deliverables

The Lab 4 submission contains:

1. Before video showing the original game.
2. After video showing the updated game.
3. Updated source code, including board.py, game.py, main.py, README.md, and requirements.txt.
4. Exported chat history in PDF format.

12. Conclusion

The Minesweeper project was improved using an iterative Vibe Coding workflow. The boundary traversal issue was fixed, difficulty modes were added, and concise action-level reveal feedback was implemented. Existing flag and win methods were inspected and tested. Python compilation and Git commands were used to validate the implementation and track changes in separate commits.